南方科技大学 SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机科学与工程系 Department of Computer Science and Engineering

Intelligent Data Analysis (CSE5002)

Lecture 6: Feature Selection and Extraction

Qi Hao

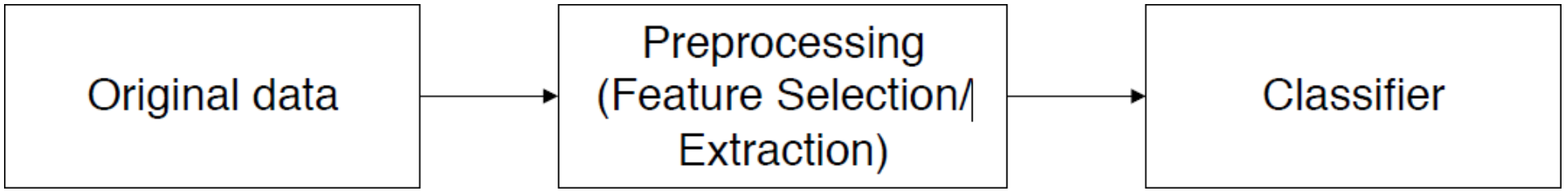
Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)
haoq@sustech.edu.cn, Office: South Tower 415
QQ Group: 1062632497
https://cse.sustech.edu.cn/faculty/~haoq/

Outline

- **Feature Analysis: Definition and Goal**
- **Feature Selection**
- **Feature Extraction**

Feature Analysis

- common name for feature selection and feature extraction
- usually for data preprocessing



three states of feature classification

- feature analysis wish to represent the data with a feature set much smaller than the original feature set, to achieve better accuracy, faster classification speed or to make the classification result more interpretable

Feature Analysis

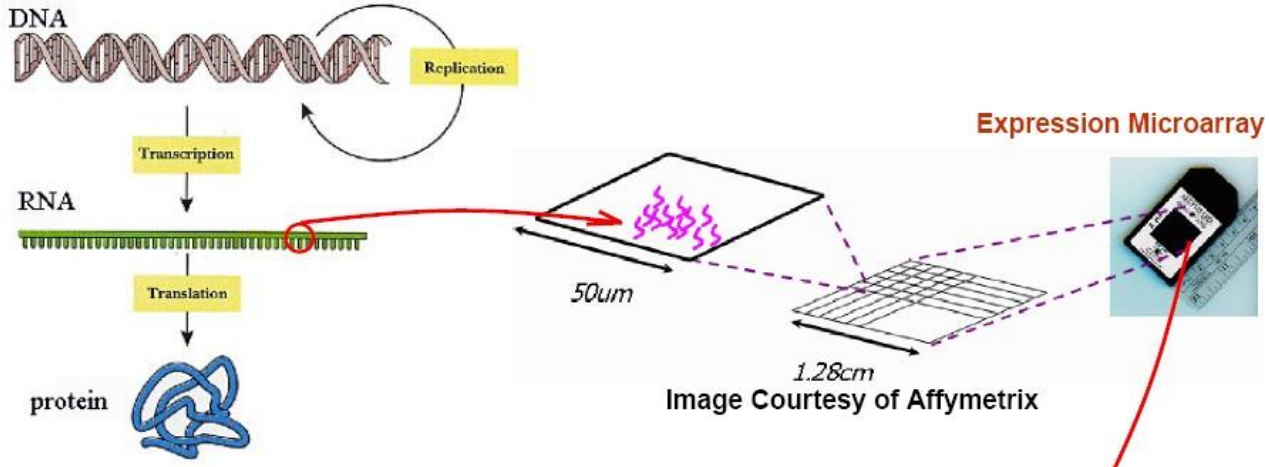
- feature analysis can improve classification accuracy, since
  - there may be noise in data
  - there are finite training examples. As the number of features increases, the generalization performance of classifiers will first increase and then decrease (curse of dimensionality)

- feature analysis can improve classification speed, since it
  - removes the requirement on storage
  - reduces the training and test time

- relationship between classification outputs and features is more clear, and can be more easily interpretable

An Example

➢ Gene Expression Microarray Analysis



- **Task:** To classify novel samples into known disease types (disease diagnosis)
- **Challenge:** thousands of genes, few samples
- **Solution:** to apply feature selection

Sample \ Gene	M23197_at	U66497_at	M92287_at	...	Class
Sample 1	261	88	4778	...	ALL
Sample 2	101	74	2700	...	ALL
Sample 3	1450	34	498	...	AML
.	.	.	.	...	.
.	.	.	.	...	.
.	.	.	.	...	.

Expression Microarray Data Set

The above figures are taken from the tutorial “Downsizing Data for High Performance in Learning—Introduction to Feature Selection Methods”, given by H. Liu and R.Stine at AAAI–2005

Outline

- **Feature Analysis: Definition and Goal**
- **Feature Selection**
  - ✓ **Filtered Method**
  - ✓ **Wrapped Method**
  - ✓ **Embedded Method**
- **Feature Extraction**

Feature Selection

- Given n training examples with each one represented by d features. We need to select a feature subset

- The search process of this feature subset is an optimization problem, with each solution represented by a d-dimensional binary vector: 0 indicates that the associated feature is not selected, 1 indicates that the feature is selected

- From this perspective, a feature selection algorithm must consists of two components:
  - how to define the “best” feature subset (requires a measurement criteria)
  - how to search through the solution space

Feature Selection

- According to the measurement criteria of feature subset, feature selection algorithm can be divided into three categories
  - filter: do not consider the performance of the follow-up classifier in the feature selection process
  - wrapper: require continuous feedback from the follow-up classifier in the feature selection process
  - embedded: the quality measurement of subset does not directly use classifiers, but some properties of classifiers instead

- For a measurement criteria, there exist many searching algorithms, for example
  - feature ranking, sequential forward/backward selection, drift forward/backward selection, evolutionary algorithm

- Typical feature selection algorithm can be seen as an iterative process. Terminate the iteration when
  - select certain number of features ($l < d$)
  - new features added bring no significant improvements in classification accuracy

Feature Selection : Filter

- Fisher's Ratio+Feature Ranking
  - calculate a Fisher's Ratio for each feature
  - rank according to Fisher's Ratio from the largest to smallest
  - choose l features with the largest Fisher's Ratio
- Calculation of Fisher's Ratio

$$FR = \frac{(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2}$$

where μ_i and σ_i are the mean and standard deviation of samples in the i-th class along this feature

- FR can be replaced by other measurements, e.g., AUC and t-test

Feature Selection —Wrapper

Example: SVM+sequential forward selection

- Initialize S as an empty set
- Let P be the set of all features
- For $i = 1 : I$ (I is the number of features to be selected)
  - for $j = 1 : r$ (r is the number of features in the set P)
    –take feature j to S, train a SVM and score this feature according to the performance (e.g., accuracy) of this SVM
    –delete j from S
  - end
  - Select the feature from P with the largest “score” and add it to S

End

Feature Selection —Embedded

- Example: SVM-RFE (sequential backward selection)
  1. Initialize S as the set of all features
  2. while $|S| > I$
     –train a linear SVM with S
     –delete the feature corresponding to the w_i of the smallest magnitude
  3. end

- SVM-RFE can delete several features one time

- Question: what will change if we do not use a linear SVM?

Outline

- **Feature Analysis: Definition and Goal**
- **Feature Selection**
- **Feature Extraction**
  - ✓ **Principal Component Analysis**
  - ✓ **Fisher Discrimant Analysis**
  - ✓ **Nonlinear Feature Extraction**
  - ✓ **Low Dimensional Data Representation**

Feature Extraction

- Different from feature selection, feature extraction transforms the original features through a function. If the transformation is linear, then it is called a linear feature extraction.

- Two classical liner feature extraction methods
  - principal component analysis (PCA) –unsupervised
  - Fisher discriminant analysis (FDA) –supervised

Feature Extraction—PCA

- Apply linear transformation to features to make the new features linearly independent to each other
  - the dependency among features can be expressed by a covariance matrix: the element in the i-th row and j-th column is the correlation between the i-th and j-th feature
  - therefore, if off-diagonal elements are zero, then all features are linearly independent

- Suppose we have training examples represented by a normalized $N \times d$ matrix X. We aim to find a matrix T such that the covariance matrix of $X' = XT$ after transformation is a diagonal matrix

- For X, eigenvectors of the associated covariance matrix meet the above requirements

Feature Extraction—FDA

- FDA also wants a linear transformation matrix T
- After transformation, we wish examples in the same class to be close, examples in different classes apart from each other
- FDA use within-class scatter matrix and between-class scatter matrix
  - within-class scatter matrix

$$S_w = \sum_{i=1}^{c} \sum_{j=1}^{N_i} p_i(\mathbf{x}_j - \mu_i)(\mathbf{x}_j - \mu_i)^{\mathsf{T}}$$

  - between-class scatter matrix

$$S_b = \sum_{i=1}^{c} p_i(\mu_i - \mu_0)(\mu_i - \mu_0)^{\mathsf{T}}$$

where μ_i is the mean of examples in the i-th class, μ_0 is the mean of all training examples

Feature Extraction—FDA

- Goal of FDA: find a matrix T to

$$maxJ := \frac{\mathrm{tr}(TS_bT^\top)}{\mathrm{tr}(TS_wT^\top)}$$

- A solution is the eigenvector $S_w{}^{-1}S_b$

- FDA can choose at most $c - 1$ independent features

Nonlinear Feature Extraction

- Nonlinear Feature Extraction
  - related to practical applications
  - implementation of PCA and FDA requires only the calculation based on inner product. Nonlinear feature extraction can be easily realized with kernel trick

Low dimensional Data Representation

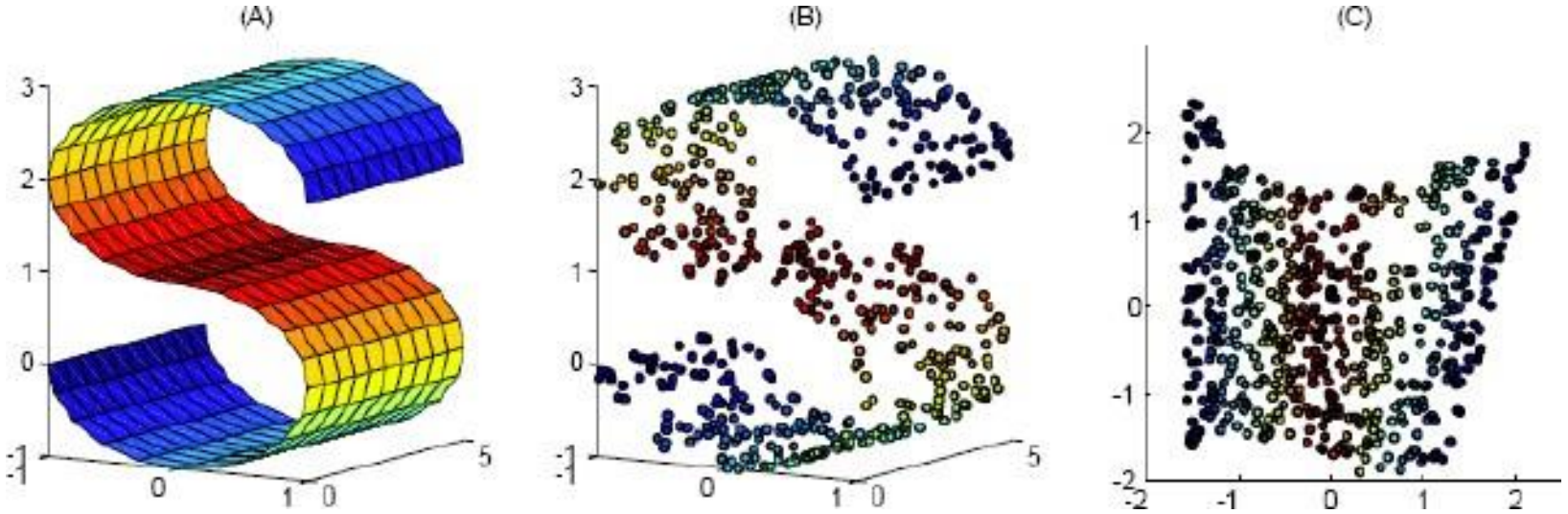
- Humans cannot visualize space above 3D. This brings great difficulties to discover properties of data distribution, as well as regular patterns underlying the data.

- Therefore, we wish to find a method to map data from a high-dimensional space to a low dimensional space, while keeping the topological property among data (e.g., similarity between two example), to simplify the acquisition of priori knowledge

- Methods
  - multi-dimensional scaling
  - locally linear embedding (LLE)

Low-dim Data Representation——LLE

- Basic assumption: the distribution of patterns in original space admits a manifold structure



- We wish to find a map in a low-dimensional space, to realize the transformation from Figure B to Figure C

LLE Algorithm

1. Calculate k-nearest neighbors for each example

for $i = 1 : N$
  compute the distance from $\mathbf{x}_i$ to every other point $\mathbf{x}_j$
  find the k-smallest distances
  assign the corresponding points to be neighbors of $\mathbf{x}_i$
end

LLE Algorithm

2. Calculation of weight matrix W

for $i = 1 : n$
  create matrix Z consisting of all neighbours of $\mathbf{x}_i$
  subtract $\mathbf{x}_i$ from every column of Z
  compute the local covariance $C = Z^T * Z$
  solve linear system $C * W = 1$ for W
  set $W_{ij} = 0$ if j is not a neighbor of i
  set the remaining elements in the i-th row of W equal to w/sum(w)
end

LLE Algorithm

3. Calculate the map Y

create sparse matrix

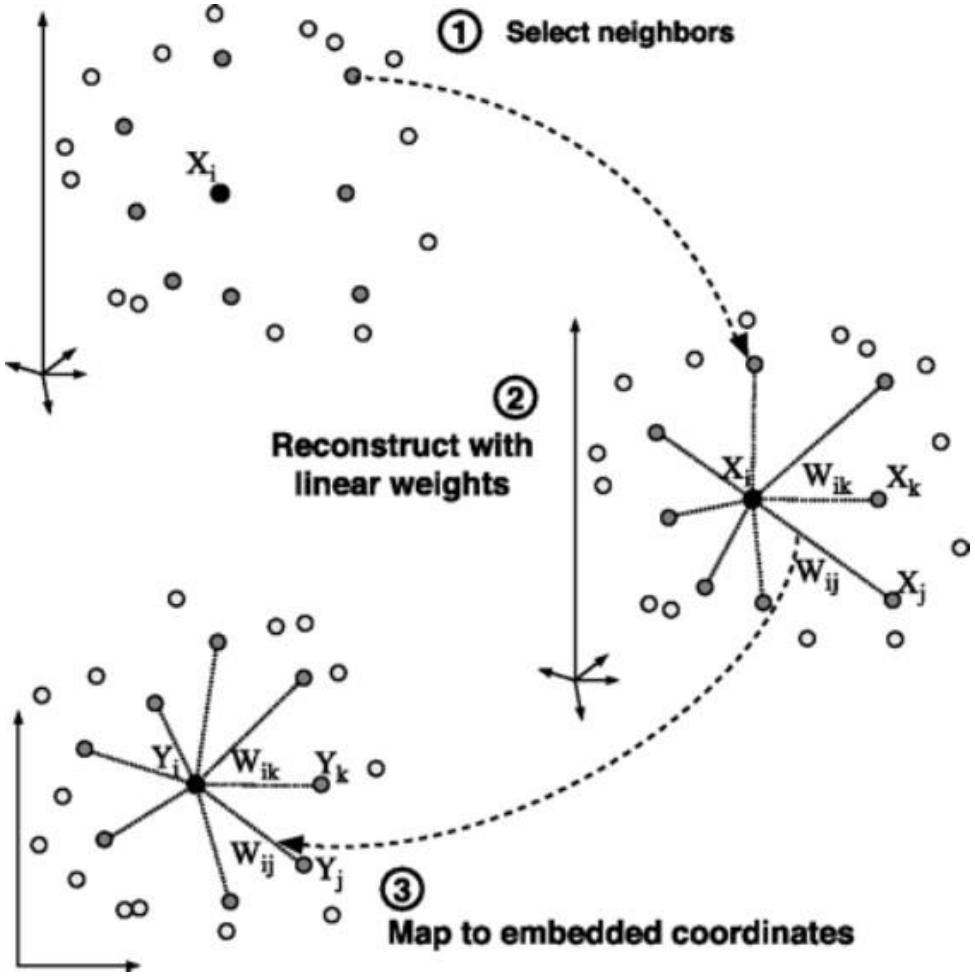
$$M = (I - W)^T(I - W)$$

find bottom $d + 1$ eigenvectors of M (corresponding to the $d + 1$ smallest eigenvalues)

set the q-th row of Y to be the $d + 1$ smallest eigenvector (discard the bottom d eigenvectors with eigenvalues of zero)

LLE Flow-chart

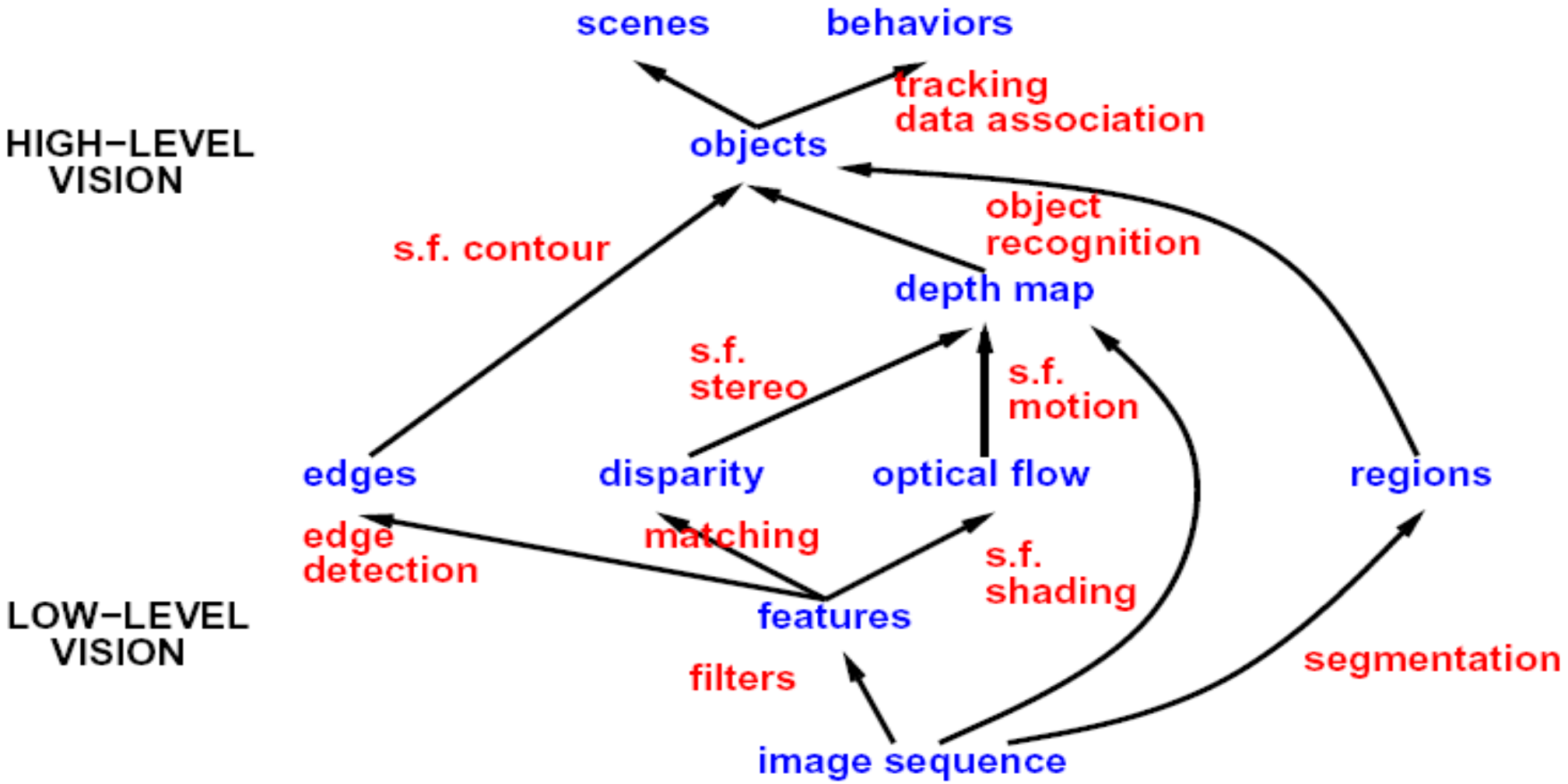
- Flow-chart of LLE



Outline

- **Image Feature Extraction**
- **Point Cloud Feature Extraction**

Vision System



Vision requires combining multiple cues

Use of Image Features

- Feature is a piece of information that describes an image or a part of it
  - ✓ Corners
  - ✓ Edges
  - ✓ Circles, ellipses, lines, blobs etc

- Features are useful to match two images
  - ✓ Find common points in two images to stitch them
  - ✓ Object recognition, face recognition
  - ✓ Structure from motion, stereo correspondence

- **Why not match pixels values?**
  - ✓ Pixel values change with light intensity, colour and direction.
  - ✓ They also change with camera orientation.

Properties of Features

- Repeatability
  - ✓Should be detectable at the same locations in different images despite changes in viewpoint and illumination
- Saliency (descriptiveness)
  - ✓Same points in different images should have similar features
  - ✓And vice-versa
- Compactness (affects speed of matching)
  - ✓Fewer features
  - ✓Smaller features

Feature Detection and Extraction

➢ Feature Detection

- ✓ Where to extract features
- ✓ Not all locations are good to extract features

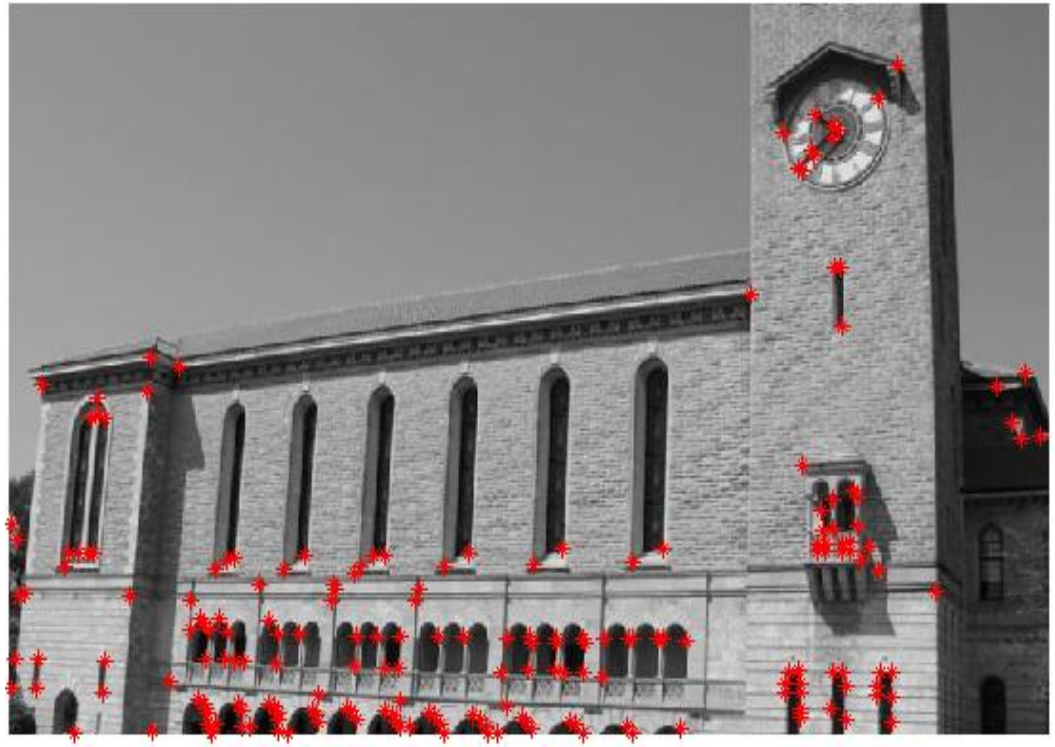
➢ Feature Extraction

- ✓ Orientation of the extraction window
- ✓ What to encode in the feature

Invariant Features

- Can you see common feature point between the two images?





Applications of Feature Matching

- Image alignment and stitching
- 3D structure from motion
- Motion tracking
- Robot navigation (SLAM)
- Image retrieval
- Object and face recognition
- Human action recognition
- Scene classification

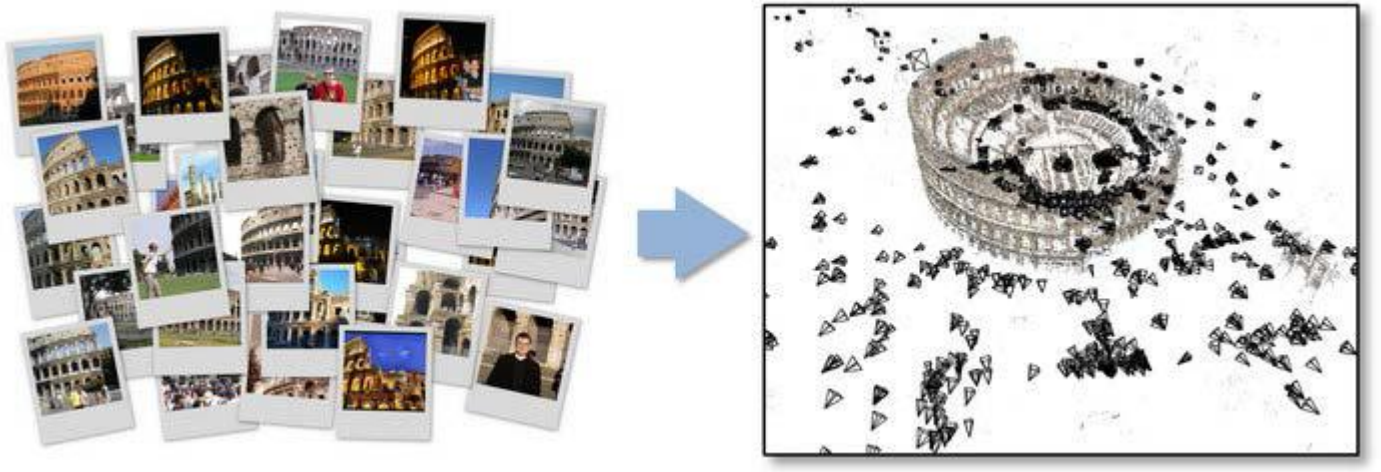
Image Switching

- How to generate panorama from multiple images?



Structure From Motion

- Building Rome in a day
- Matching feature points in unordered image collections
- Software by Noah Snavely, Cornell University



Feature Detectors

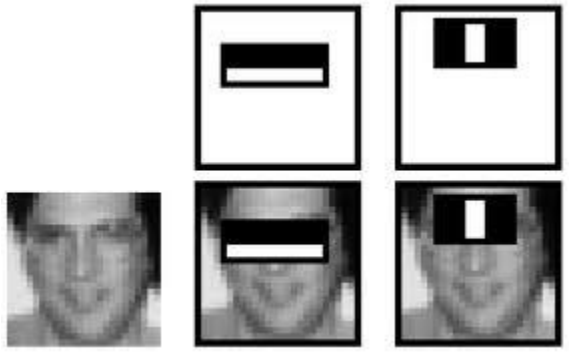
Feature	Detection	Extraction	OpenCV	Published
Harris	Yes	No	Yes	1988
KLT	Yes	No	Yes	1994
LBP	No	Yes	Yes	1994
SIFT	Yes	Yes	Yes	IJCV 2004
FAST	Yes	No	Yes	ECCV 2006
SURF	Yes	Yes	Yes	CVIU 2008
BRIEF	No	Yes	~	ECCV 2010
ORB	Yes	Yes	Yes	ICCV 2011
BRISK	Yes	Yes	Yes	ICCV 2011
FREAK	Yes	Yes	Yes	CVPR 2012

Examples of Image Feature Extraction

- Haar features
- HOG: Histogram of Oriented Gradients
- LBP: Local Binary Patterns
- Harris Corners
- Scale Invariant Feature Transform (SIFT)

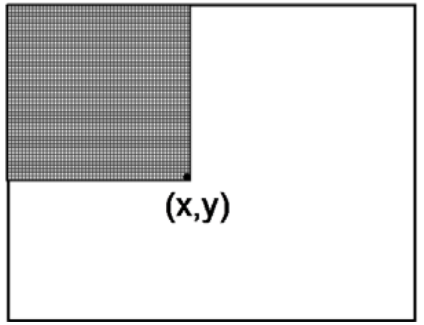
Haar Features

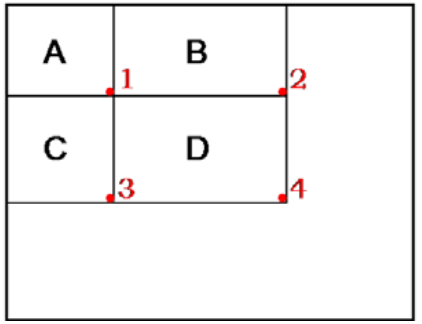
- Haar wavelets but very simple and fast to calculate
- Sum of white region pixels minus the sum of black region pixels
- Can be calculated in real time from integral image
- A large number of weak classifiers make decisions based on Haar features
- When combined, they become a strong classifier



Sum of pixels in D = 4 – 2 – 3 + 1

Integral image





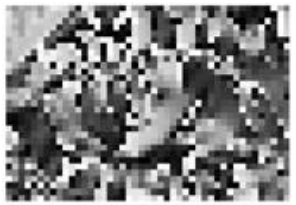
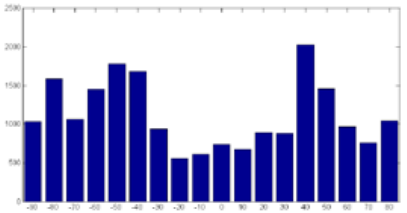
Haar Feature Application

- Realtime XXX detection
- Where XXX can be
  - ✓ Faces
  - ✓ Stop signs
  - ✓ Fish
  - ✓ Any object
- Using C++ OpenCV library, you can train a Haar detector for any object
- You can also do it using Matlab Computer Vision System Toolbox

Histogram of Oriented Gradients (HOG)

- Calculate gradient magnitude and orientation at each pixel
- Divide orientations into N bins e.g. 360 degrees can be divided into 18 bins
- The gradient magnitude of each pixel is then voted into the bin corresponding to its orientation
- Improved variants
  - Distribute the gradient magnitude among neighbouring bins (soft voting)
  - Divide image into cells, compute HOG for each cell and concatenate them
  - Use a Gaussian window to weight the gradient magnitudes
  - Normalize the final HOG to unit magnitude



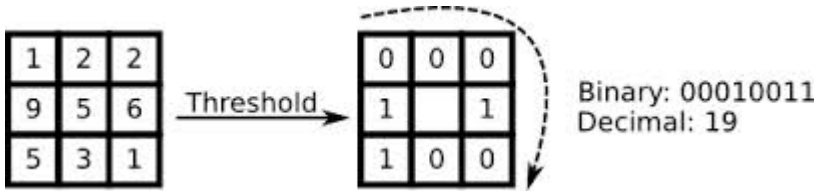



HOG Applications

- Can be used for detection of objects or actions
- Has shown very good results for pedestrian detection
- Has shown good results for action recognition

Local Binary Patterns (LBP)

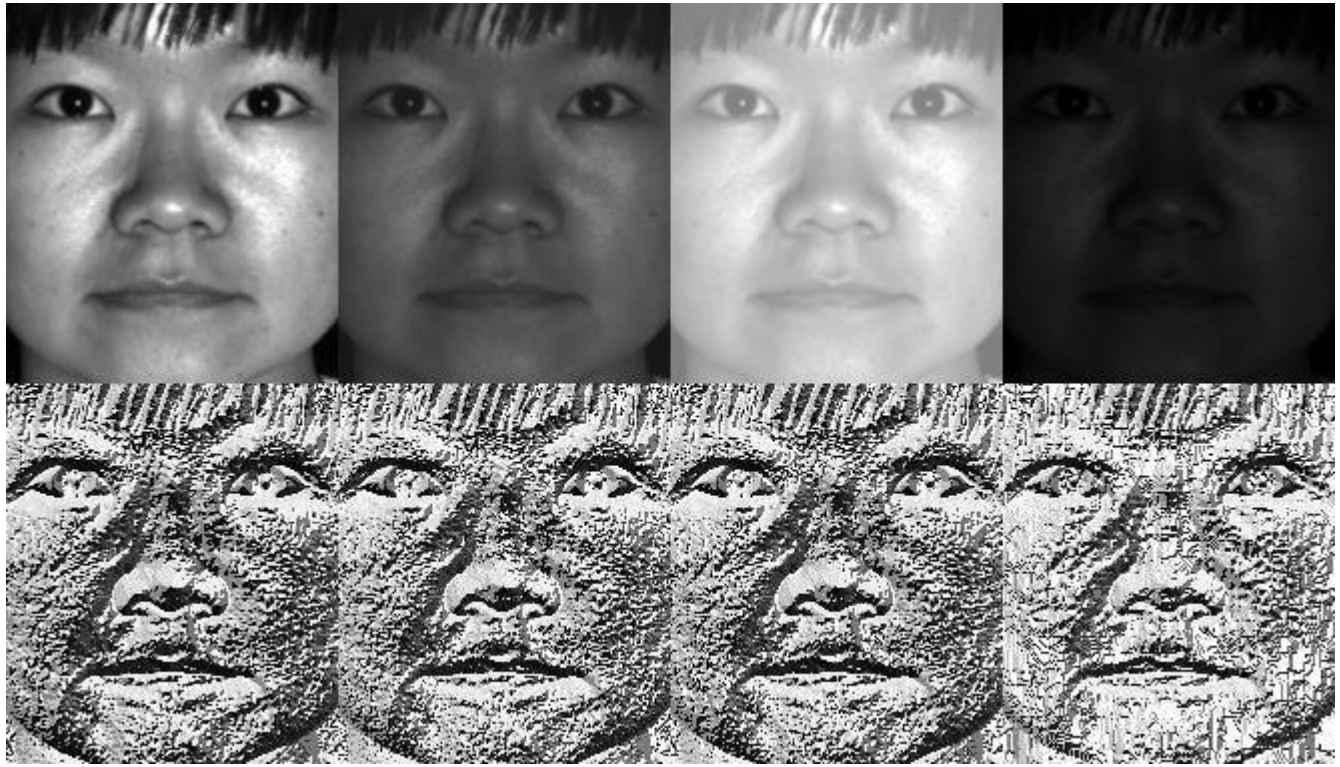
- Compare 8-connected neighbourhood with center pixel
- If pixel > center, replace with ‘1’ else ‘0’
- Construct a binary number by going clockwise
- Replace the center pixel with the decimal value of the binary number



- Binary number is sensitive to starting point – LBP is not rotation invariant
- Rotate binary string to minimize decimal value for rotation invariance
- Minor changes in illumination can change the decimal value
- Partition image into cells and construct LBP histograms in each cell

LBP Example

- Notice how LBP feature are illumination invariant



Global or Local Feature Extraction

- LBP, HOG and Haar features do not have an embedded interest point detection

- Such features must be calculated at each pixel or globally from the overall image
  - ✓ Sensitive to occlusions
  - ✓ Sensitive to orientation
  - ✓ Computationally expensive

Point of Interest (POI) Detection

- A corner is a point around which the gradient has two or more dominant directions

- Corners can be repeatably detected under varying illumination and view point changes

Local Feature Extraction

- We have found interest points, how do we extract features?
- Take a square region around the corner point. But how big?
- What if the images to be matched have different scales?
- What if the object is rotated in the second image?

Different Scales

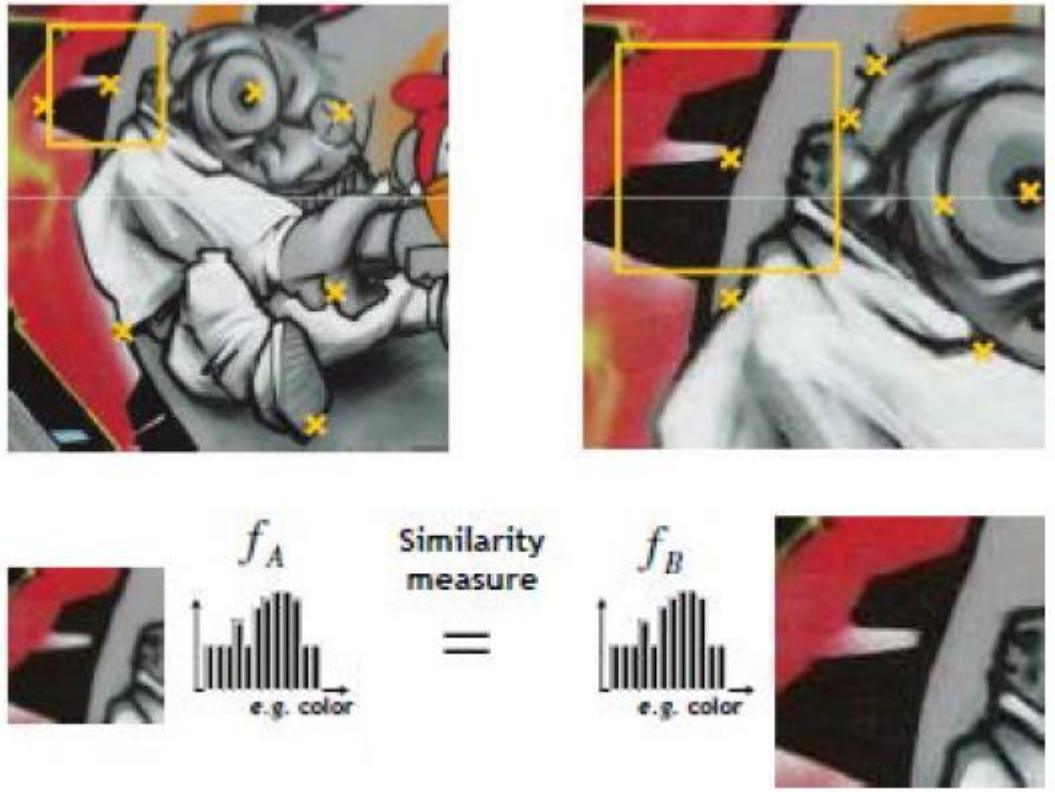








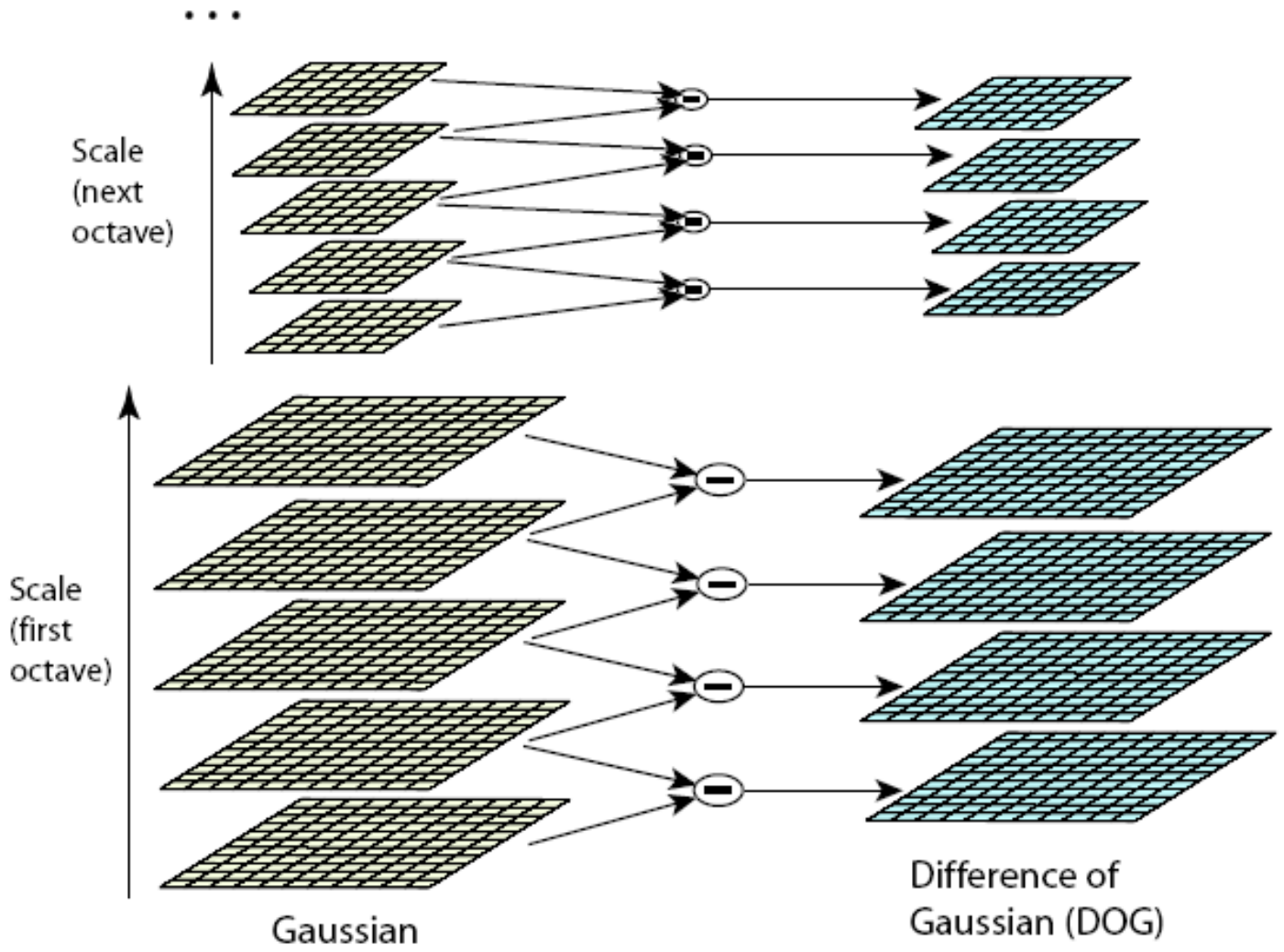
Different Window Sizes



Automatic Scale

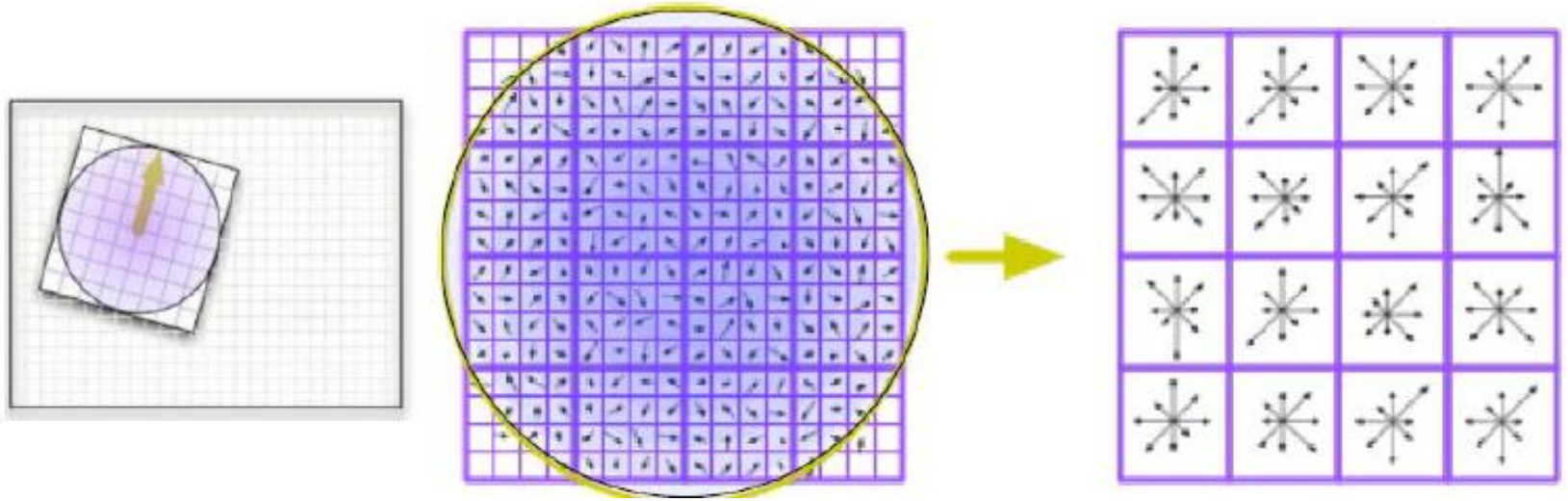
- What would be the scale for Harris corner?
- Can we efficiently find interest points and their scales together?
- Such points are said to be scale invariant.
- The search for these points is referred to as scale space search.

Gaussian Scale Pyramid



Scale Invariant Feature Transform (SIFT)

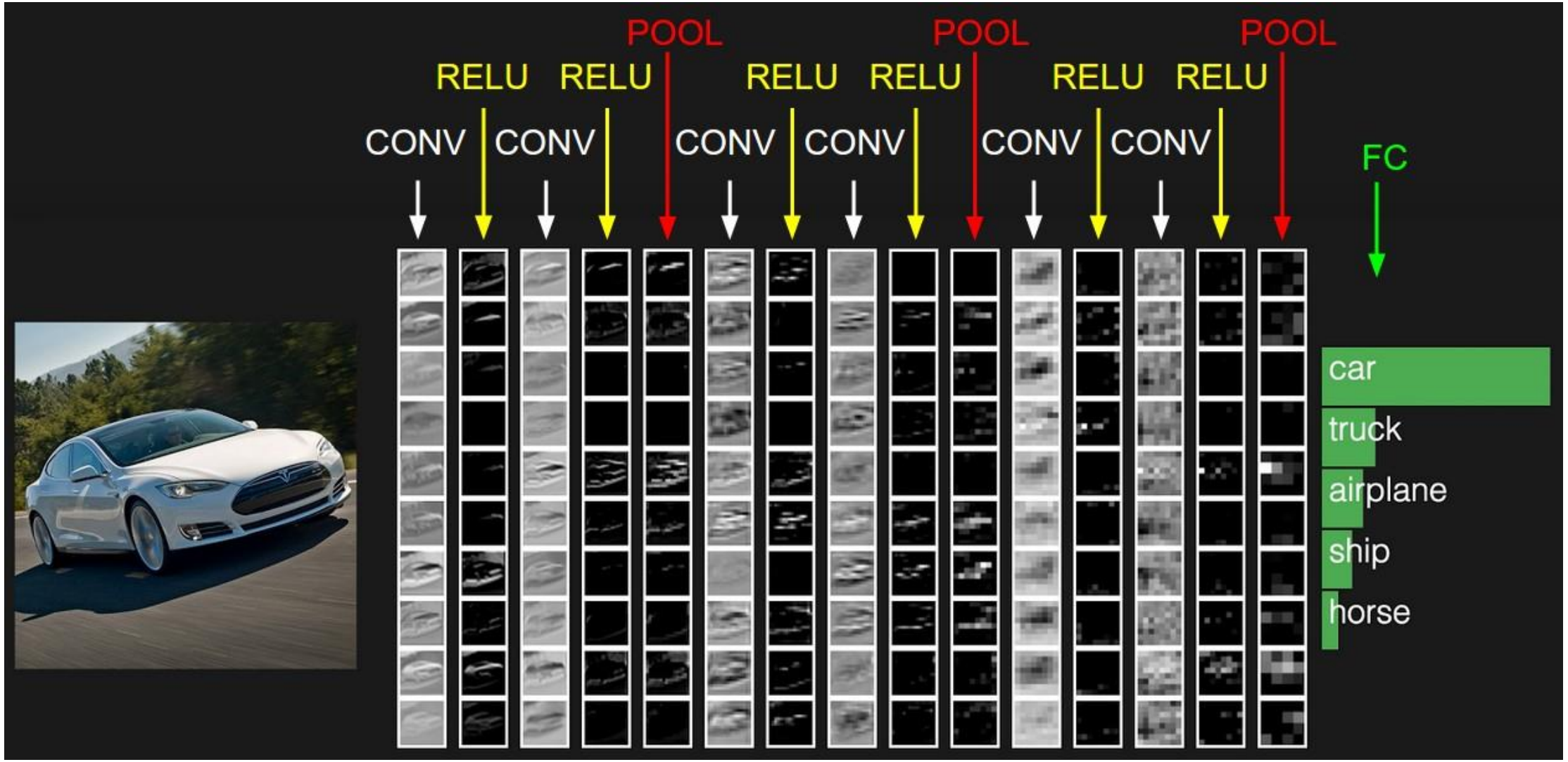
- Take the region around a keypoint according to its scale
- Rotate and align with the previously calculated orientation
- 8 orientation bins calculated at 4x4 bin array
- 8 x 4 x 4 = 128 dimension feature



SIFT Properties

- SIFT is a 128 dimensional vector
- For robustness to illumination, normalize the feature to unit magnitude
- To cater for image saturations, truncate the feature to 0.2
- Renormalize to unit magnitude
- SIFT properties
  - ✓ Repeatable keypoints
  - ✓ Scale invariant
  - ✓ Rotation invariant
  - ✓ Robust to viewpoint
  - ✓ Robust to illumination changes

CNN Based Feature Extraction



CNN Libraries

- Pytorch
  - ✓ C++ framework with Python wrapper
  - ✓ Open source
  - ✓ Meta support
  - ✓ Easy for research
- Tensorflow
  - ✓ C++ framework with Python wrapper
  - ✓ Open source
  - ✓ Google support
  - ✓ Becoming industry standard
- Mindspore
  - ✓ C++ framework with Python wrapper
  - ✓ Huawei support
  - ✓ Chinese applications

Transfer Learning

- **ConvNet as a fixed feature extractor:**
  - ✓ Take a pretrained ConvNet, remove the last fully-connected layer,
  - ✓ Then treat the rest of the ConvNet as a fixed feature extractor for the new dataset

- **Fine-tuning the ConvNet:**
  - ✓ The second strategy is to not only replace and retrain the classifier on top of the ConvNet on the new dataset,
  - ✓ but to also fine-tune the weights of the pretrained network by continuing the backpropagation

- **When and how to fine-tune?**
  - ✓ Size of the new dataset
  - ✓ Similarity to the original dataset

CNN Limits

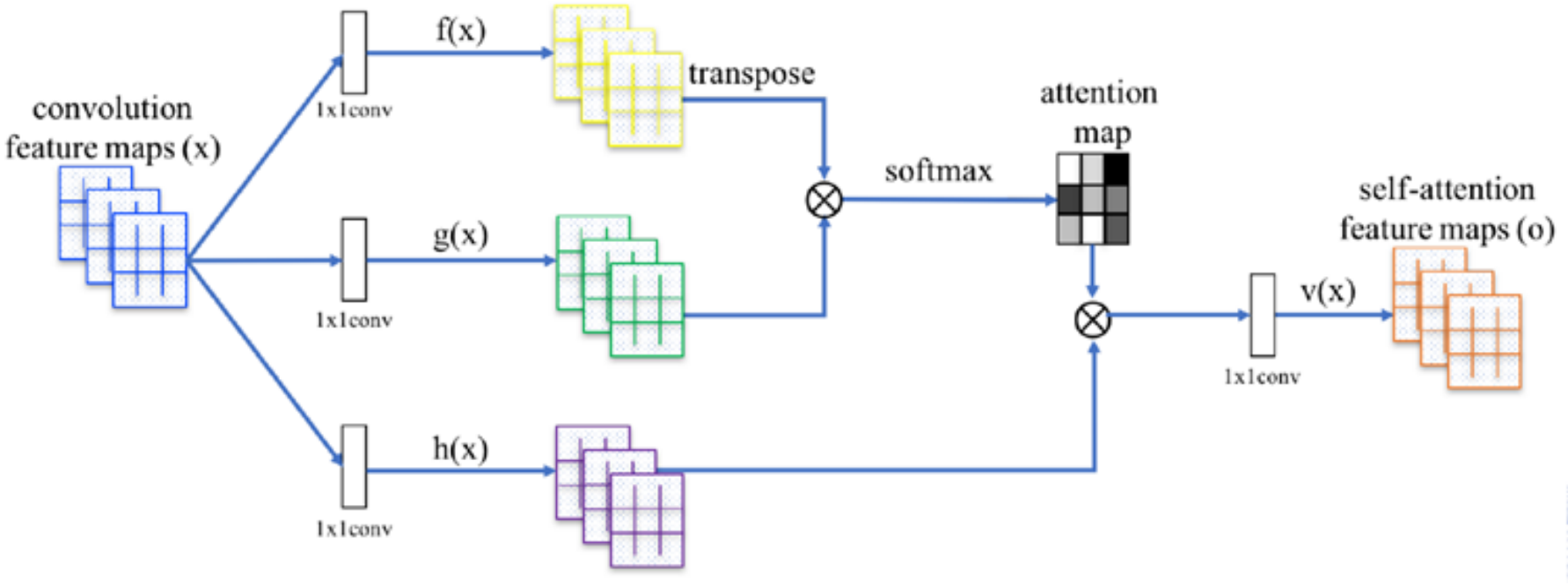
- Convolutional Neural Networks (CNNs) at each layer employ moving convolution kernel (2D filter) windows; Those learned feature detectors are **local operators** and cannot estimate the distant image patch correlations

- CNN features do not consider important **spatial hierarchies** between objects; only local interactions are considered in each convolutional layer and CNNs detect parts with no sense of the whole

- Most CNN architectures leverage **pooling** for increasing the receptive field of higher-level layers kernels, allowing them to capture higher-level features on large image regions, but **pooling** may lead to severe information loss

- CNNs rely on the assumptions of **locality** and **stationarity** governing the 2D image signal

CNN Augmented with Attention



- Operator $\otimes$ denotes matrix multiplication.
- $f(X), g(X), h(X)$ can be considered as queries Q, keys K and values V, respectively.
- Softmax is performed row-wise [ZHA2018].

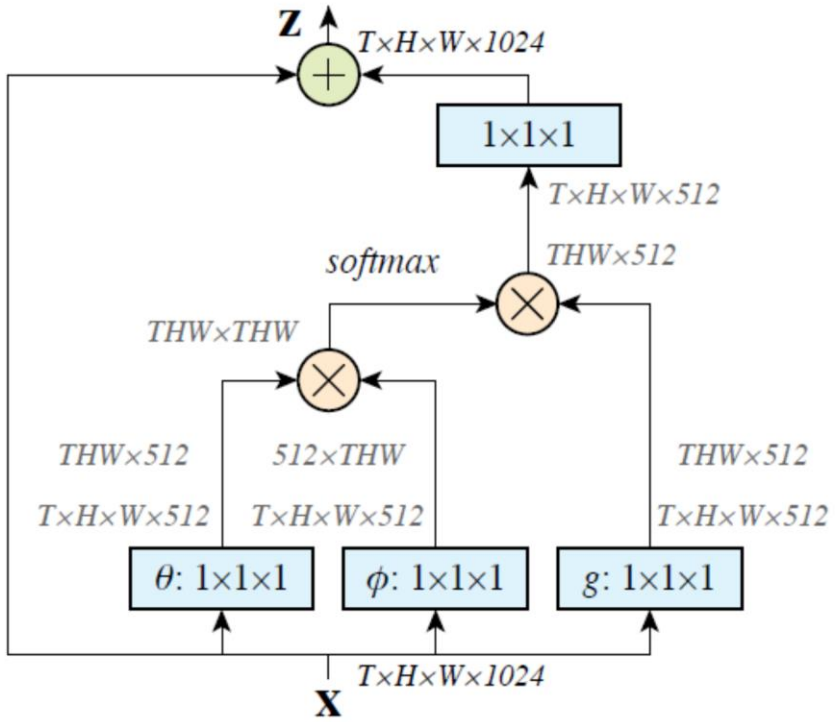
CNN Augmented with Attention

Output spatiotemporal feature map

$Z \in \mathbb{R}^{THW \times C_I}$

is generated from

$X \in \mathbb{R}^{THW \times C}$:



Spatiotemporal non-local block. ⊗ and ⊕ denote matrix multiplication and addition respectively [WAN2018].

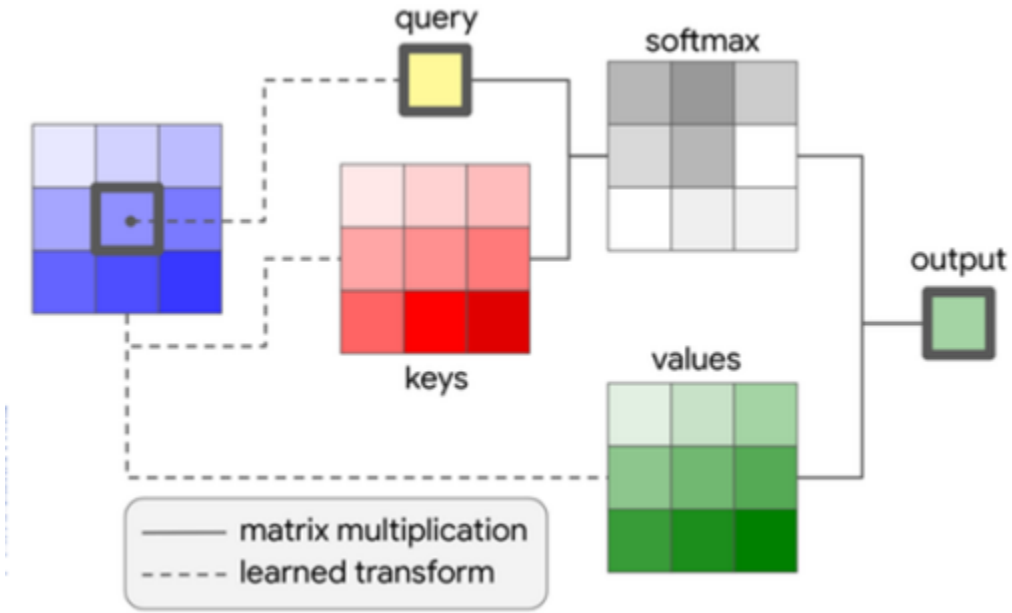
CNN Augmented with Attention



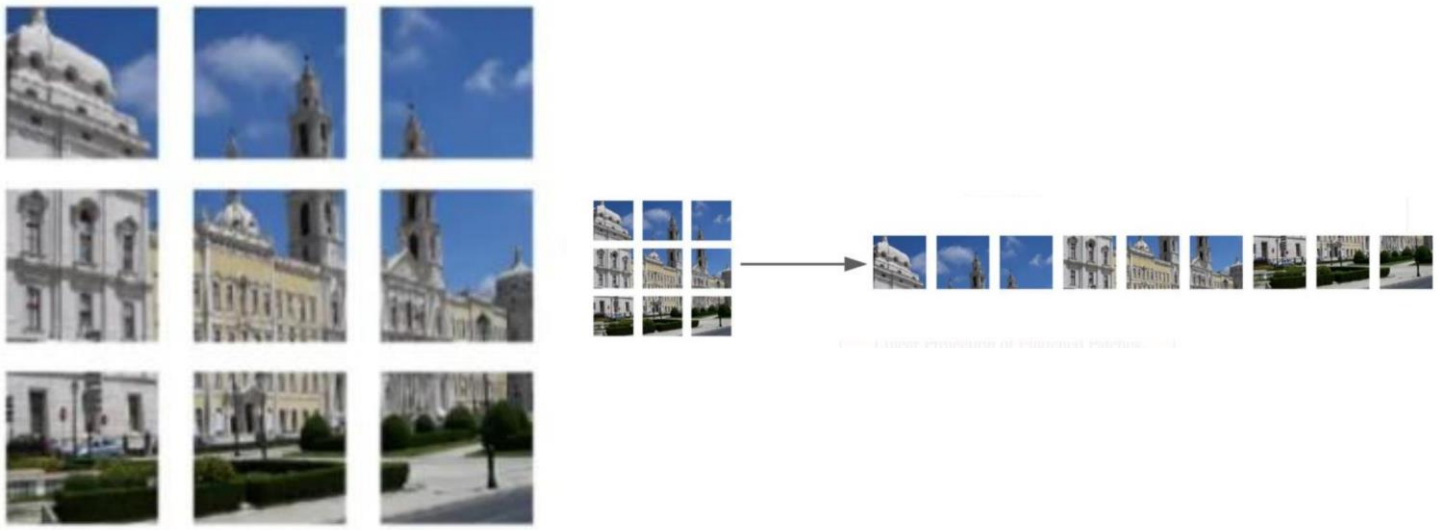
Correlations captured by non-local blocks [WAN2018].

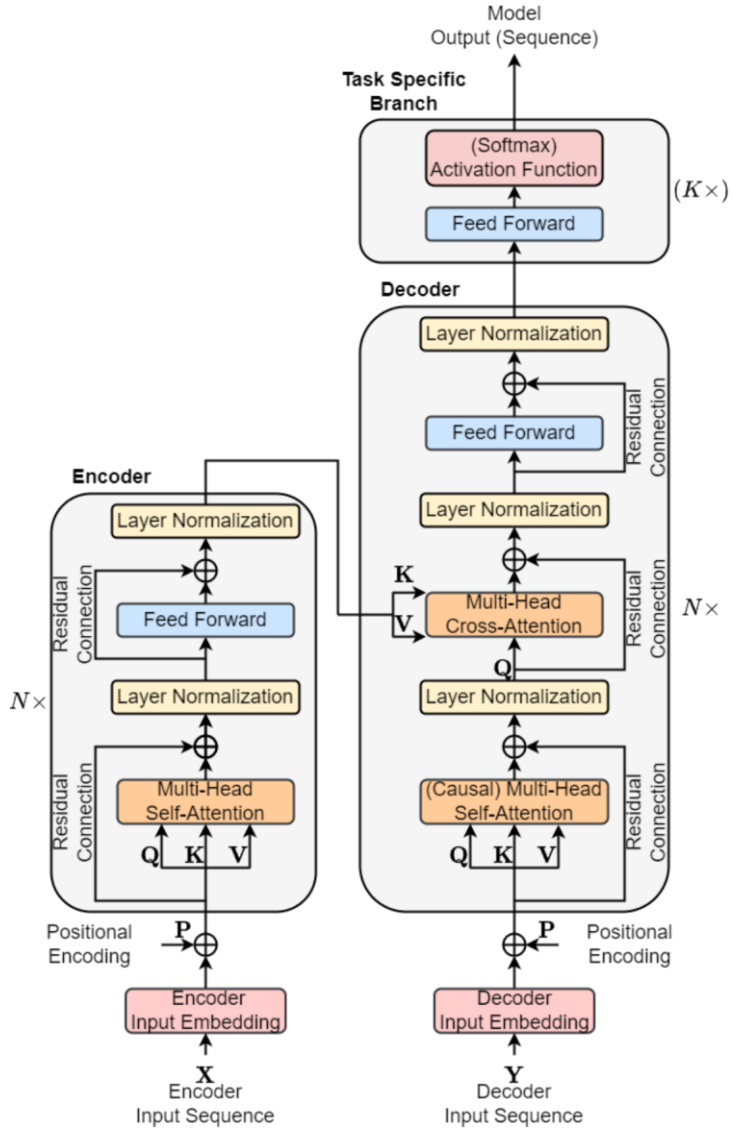
Attention is All You Need

- Attention can be used as stand-alone primitive for vision models instead of serving just as augmentation on top of convolutions.
- Attention kernel slides accross different image regions.



Transformer Architecture





- Transformer was originally designed for neural sequence transduction
- It has an encoder-decoder structure followed by one or more task specific branches

Transformer Architecture: Encoder

- The encoder consists of a stack of N identical blocks.
- Each block has two sub-layers:
  - ✓ A multi-head self-attention module.
  - ✓ A position-wise fully connected feed-forward network
- Residual connection is employed around each sub-layer followed by layer normalization

Transformer Architecture: Decoder

- The decoder also consists of a stack of N identical blocks.
- Each block has three sub-layers.
  - ✓ A (***causal***) ***multi-head self-attention module***. Optionally a mask is employed to prevent current data point from attending subsequent ones.
  - ✓ A ***multi-head cross-attention module*** between encoder and decoder sequences.
  - ✓ A ***position-wise*** fully connected ***feed-forward*** network.
- Again, residual connection and layer normalization are applied around each sub-layer.

Transformer Architecture

- They can have multiple heads, facilitating both parallelization and attention to different regions.

- Transformer encoder and decoder can have multiple layers.

Transformer Architecture

Scaled dot-product attention

Three new matrices $Q \in \mathbb{R}^{L \times d_k}$ (***queries***), $K \in \mathbb{R}^{L_I \times d_k}$(***keys***), $V \in \mathbb{R}^{L_I \times d_v}$ (***values***) are generated:

$$Q = ZW_Q + 1_{L \times 1} b_Q, \quad W_Q \in \mathbb{R}^{d_m \times d_k}, b_Q \in \mathbb{R}^{d_k},$$
$$K = Z_I W_k + 1_{L_I \times 1} b_k, \quad W_k \in \mathbb{R}^{d_m \times d_k}, b_k \in \mathbb{R}^{d_k},$$
$$V = Z_I W_v + 1_{L_I \times 1} b_v, \quad W_v \in \mathbb{R}^{d_m \times d_v}, b_v \in \mathbb{R}^{d_v},$$

by linearly transforming two matrices $Z \in \mathbb{R}^{L \times d}$ and $Z_I \in \mathbb{R}^{L_I \times d}$, where $L \neq L_I$,

- Learnable parameters: $W_Q, W_k\ W_v, b_Q, b_k, b_v$.
- In the ***original model***, it is arbitrarily chosen that $d_k = d_v$.

Transformer Architecture

Attention is an averaging of ***values***, associated to ***keys*** matching to specific ***queries***.

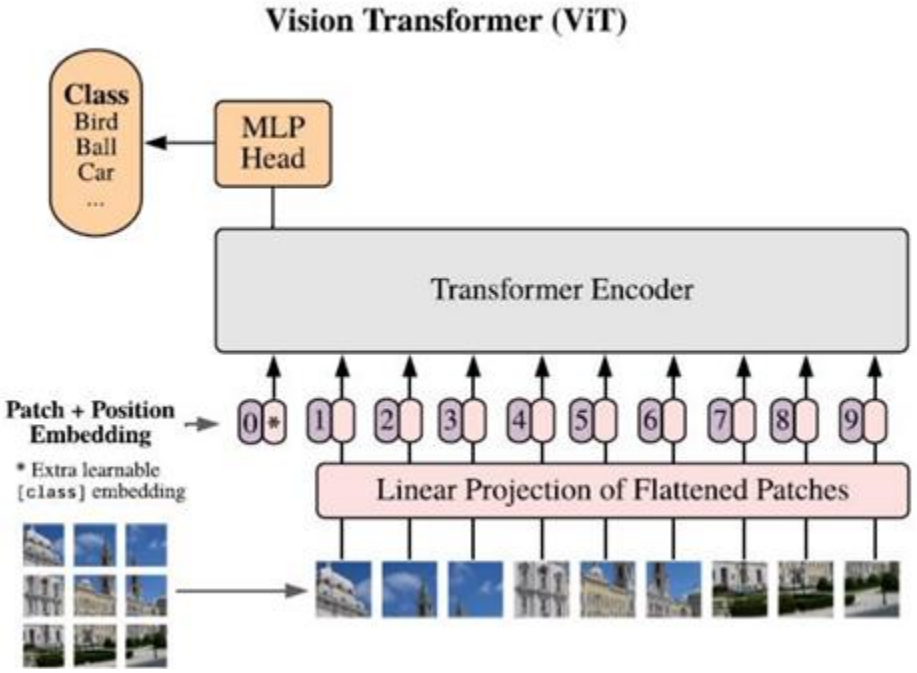
In ***cross-attention,*** each data point of sequence *x* attends to all data points of sequence Z in order to compute a new representation of sequence Z:

$$\mathbf{Y} = Softmax\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}.$$

The ***row-wise Softmax operator*** renders a probability distribution, representing the normalized correlation scores of each query to all the keys.

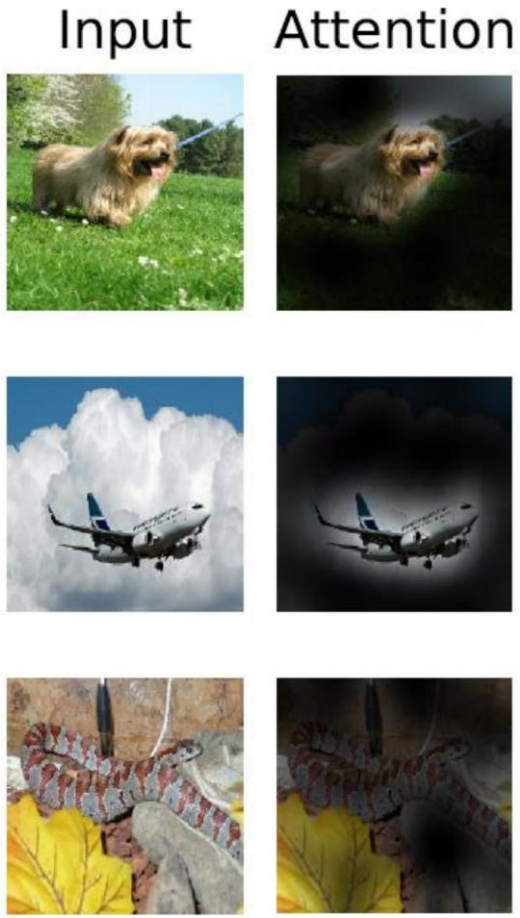
Vision Transformer (ViT)

- In ViT, an image is split into non overlapping patches. The sequence of linear embeddings of these patches is provided as input to a Transformer encoder.
- The model is trained on image classification task in supervised fashion
- Typically, ViT is pre-trained on large datasets and fine-tuned on downstream tasks



Vision Transformer (ViT)

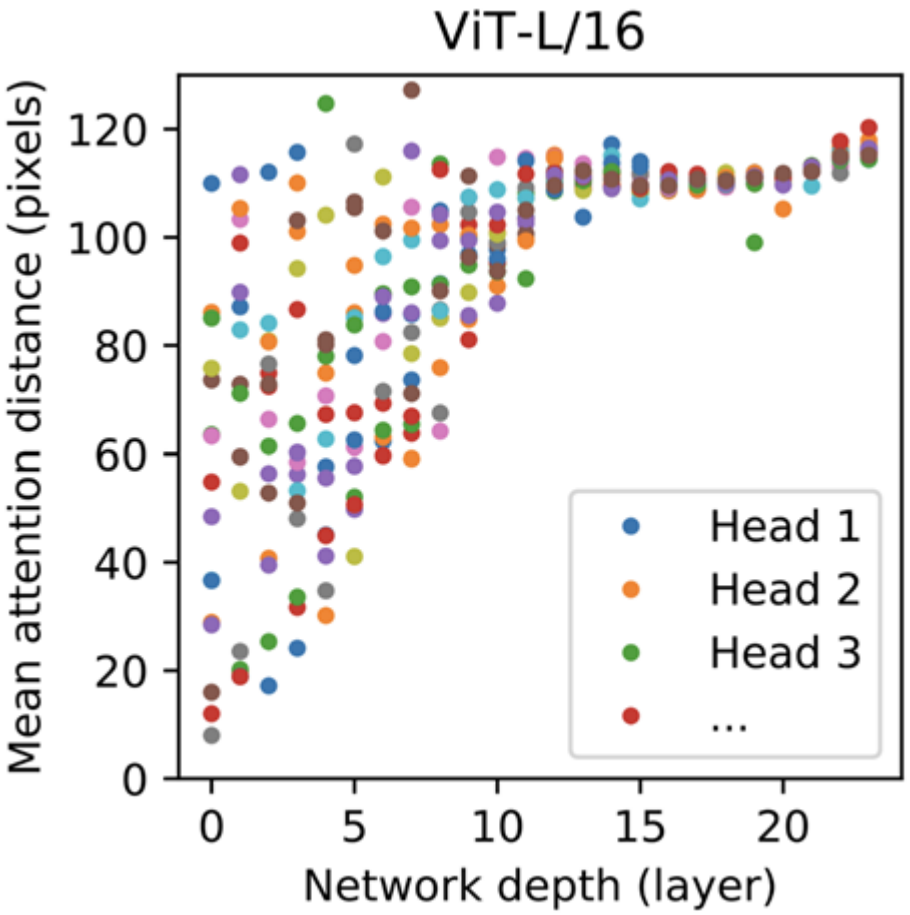
- For extracting the distribution of class probabilities, the output token attends to semantically relevant image regions.



Attention of the output token to the input space [DOS2021].

Vision Transformer (ViT)

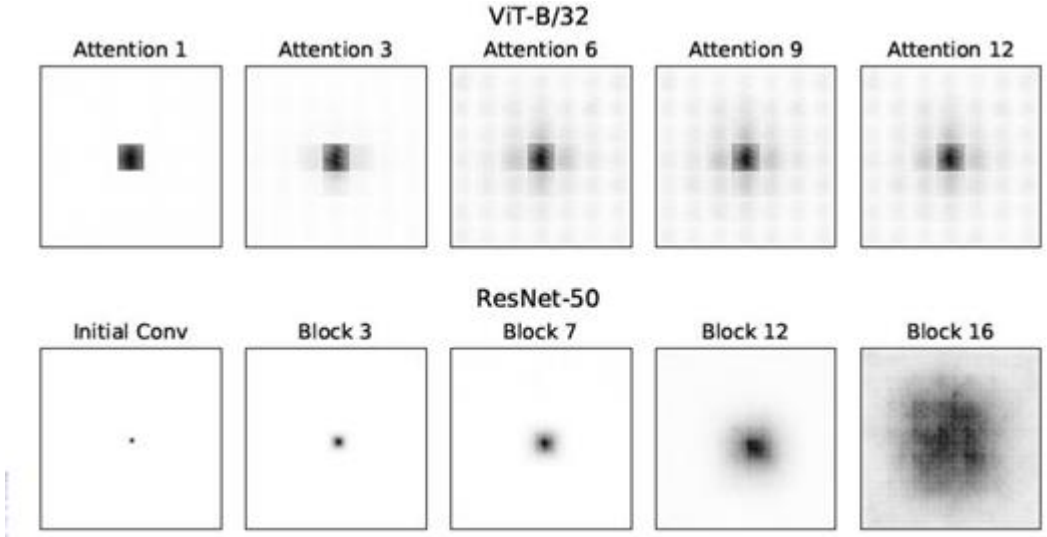
- The attention distance increases with network depth.
- It can be considered to be analogous to the receptive field in CNNs.
- Globally, the ViT model attends to image regions that are semantically similar for classification.



Distance of attended area by head and layer

Vision Transformer (ViT)

- ViT attention distances shift from local to global when moving deeper in the network.
- ResNet effective receptive fields are highly local and grow gradually, when moving deeper in the CNN network.

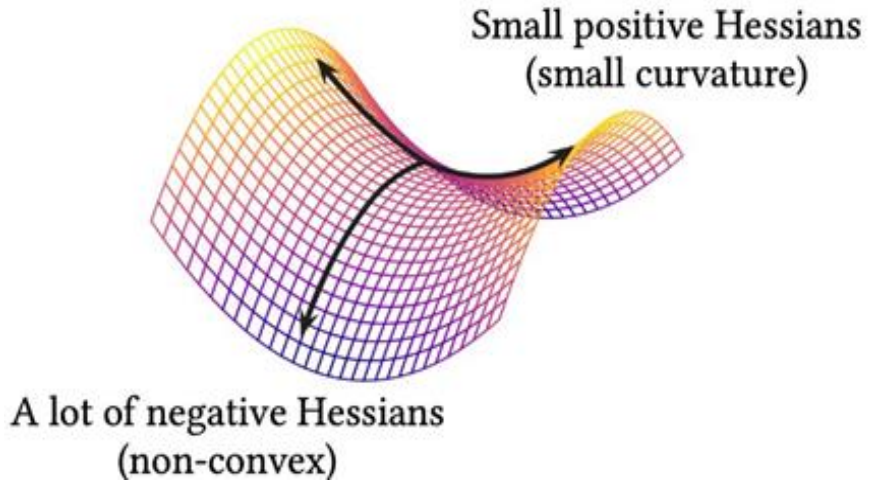


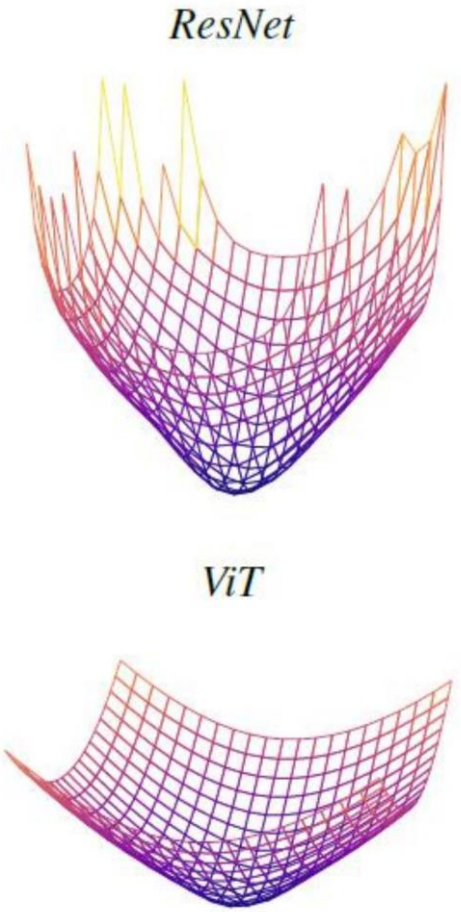
ViT attention distance vs ResNet receptive field

Vision Transformer (ViT)

- When trained on large datasets, multi-headed ViT attention flattens the loss function, leading to better performance and generalization

- When trained on small datasets, multi-headed ViT attention allows negative Hessian eigenvalues, leading to non- convex loss function forms.

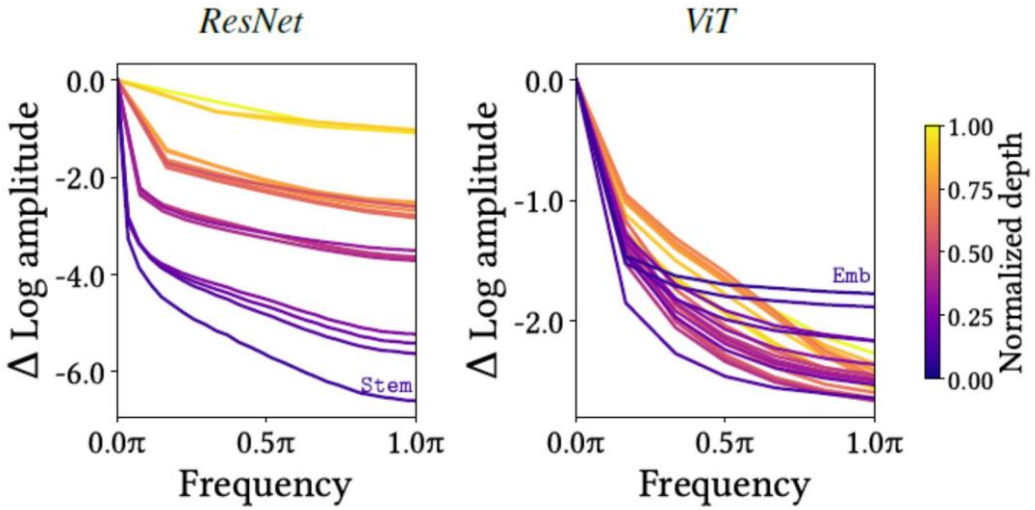




Negative log likelihood loss + ℓ_2 regularization

Vision Transformer (ViT)

- Fourier analysis of feature maps can show that Multi-headed Self- Attention (MSA) modules dampen high signal frequencies, while convolutional kernels tend to amplify them.
- Thus, MSA layers are homogeneous region-biased, while convolutional ones are texture- biased.

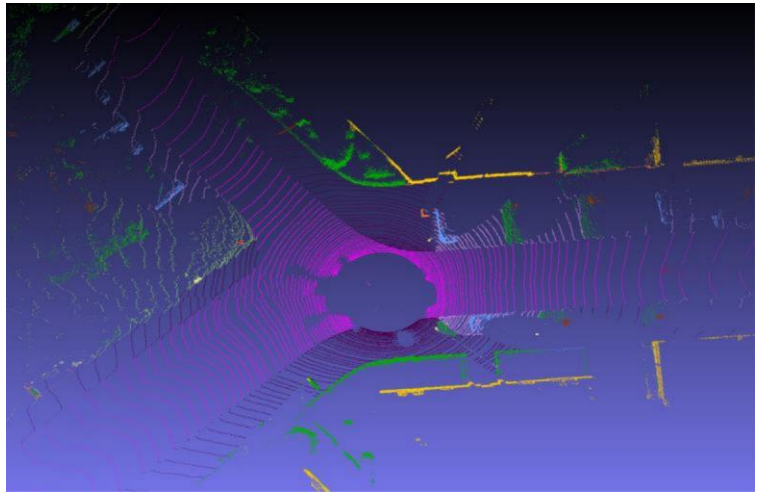


Relative log amplitudes of Fourier transformed feature map. Δ log amplitude of high-frequency signals is the difference between the log amplitude at normalized frequency 0.0π and at 1.0π [PAR2022].

Outline

- Image Feature Extraction
- Point Cloud Feature Extraction

Applications of 3D Point Clouds



Autonomous driving



Robotics

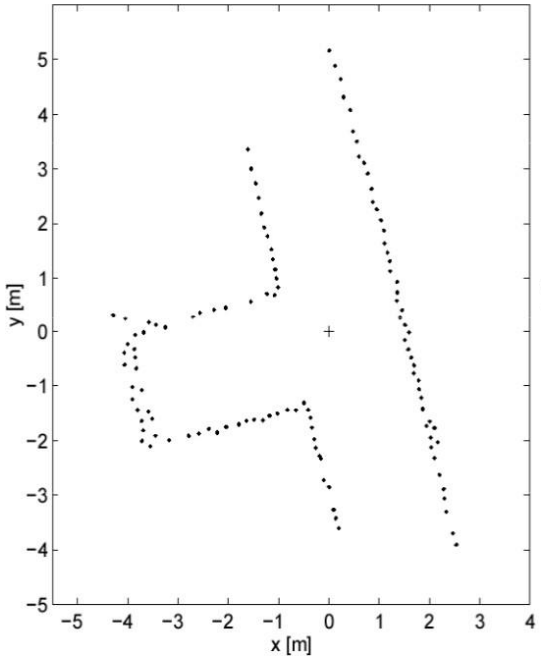


AR/VR

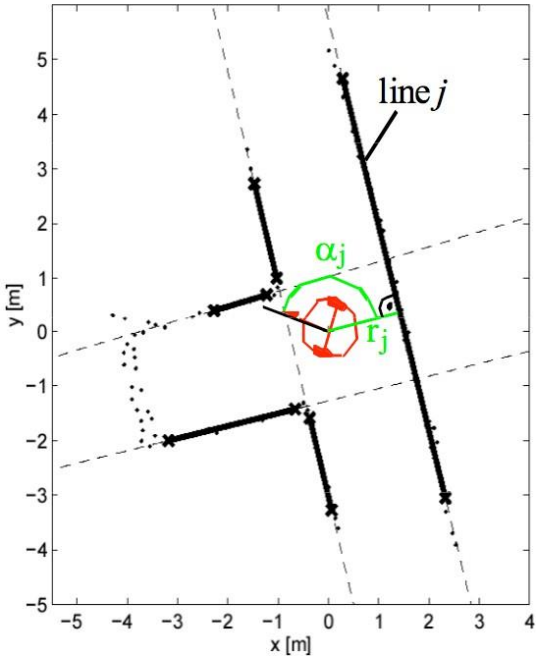
Classical Point Cloud Feature

- Point-cloud feature include the tangent vector and offset

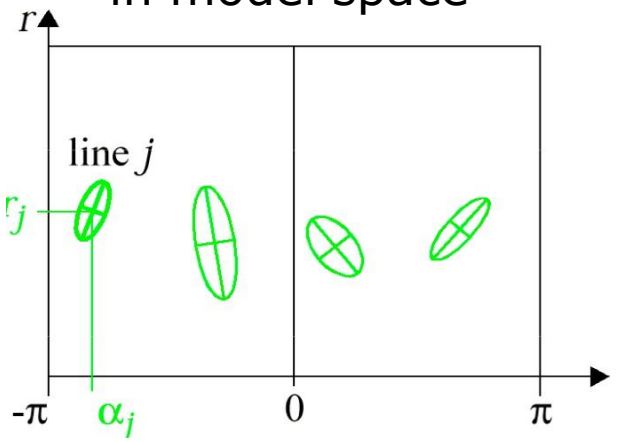
Raw laser range data



Extracted lines



Extracted lines in model space



Hessian line model

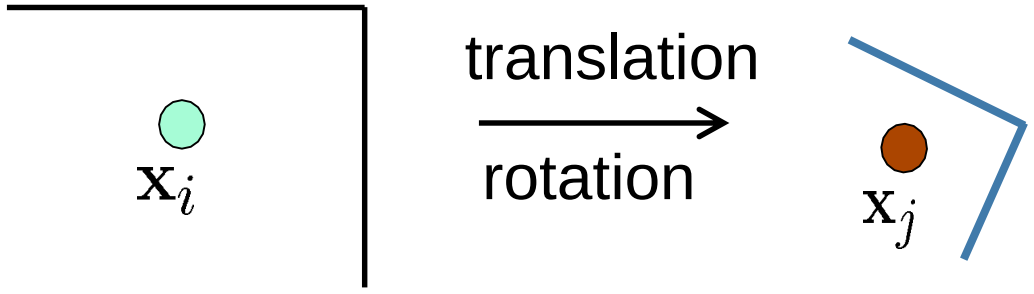
$$x\ cos(\alpha) + y\ sin(\alpha) - r = 0$$

$$\mathbf{z}_k = \begin{bmatrix} \alpha \\ r \end{bmatrix}$$

$$R_k = \begin{bmatrix} \sigma_\alpha^2 & \sigma_{\alpha r} \\ \sigma_{r\alpha} & \sigma_r^2 \end{bmatrix}$$

Classical Point Cloud Feature

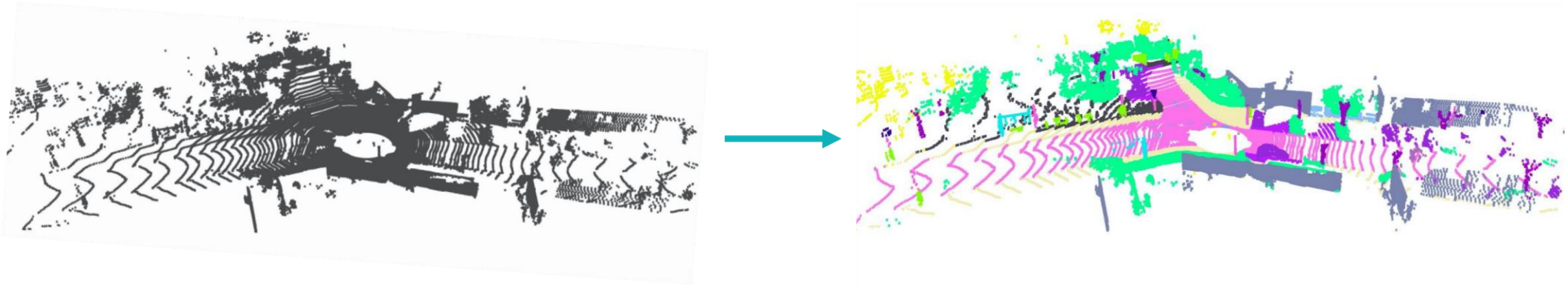
- The LiDAR observes the same part of the environment from $\mathbf{x}_i$ and from $\mathbf{x}_j$



Measurement from $\mathbf{x}_i$ Measurement from $\mathbf{x}_j$

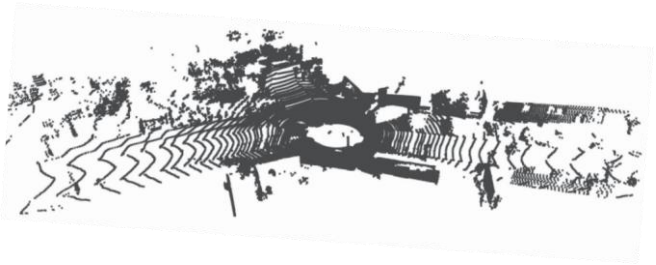
Understanding 3D Point Clouds

- Point-cloud understanding through semantic segmentation

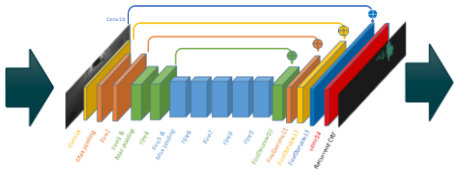


Understanding 3D Point Clouds

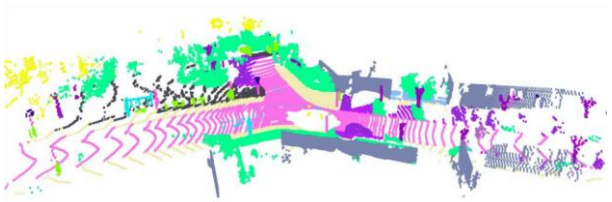
- Accuracy: essential for applications such as autonomous driving, AR/VR, etc.
- Efficiency: Real-time speed, low energy on embedded processors



LiDAR point cloud



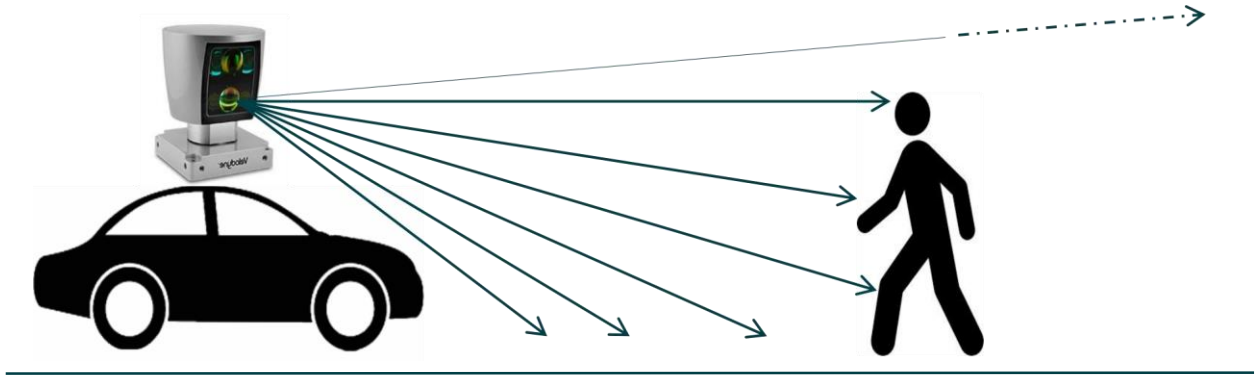
Deep Neural Net

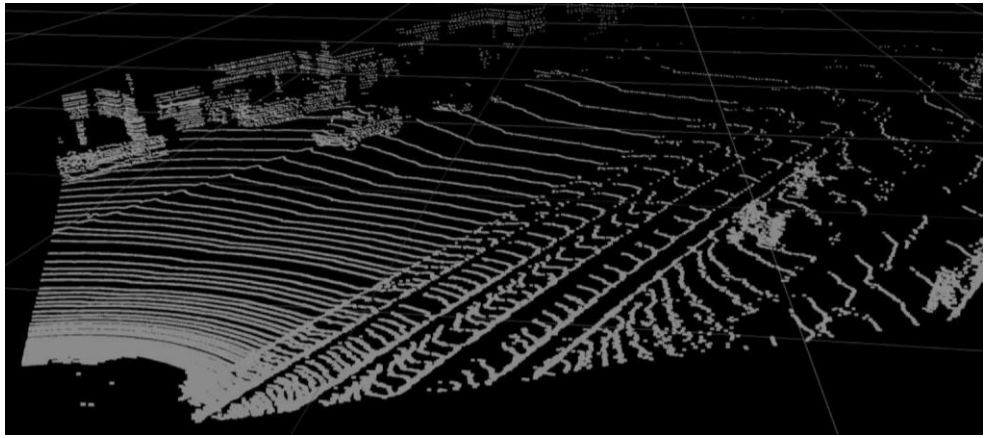


Point-wise object labels
(car, person, etc.)

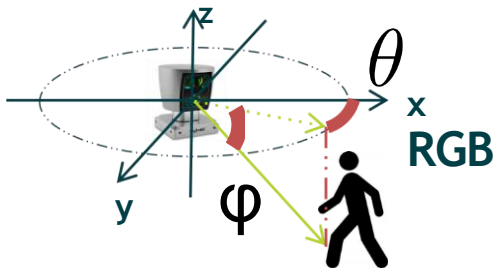
LiDAR

- LiDAR (**Li**ght **D**etection **A**nd **R**anging) is an important sensor for autonomous driving
- **An example: a Velodyne-64 LiDAR**
  - Emitting lasers, and measure distances through time-of-flight
  - Emitting 64 rays per pulse, 2000 pulses per rotation, and 10 rounds per second



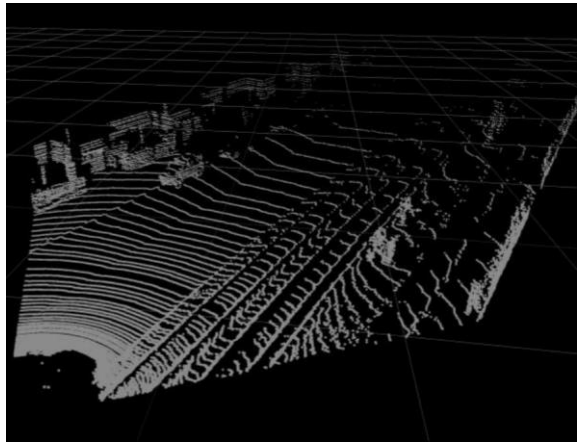


Projecting a 3D Point Cloud to a 2D sphere



RGB





Spherical projection



Intensity

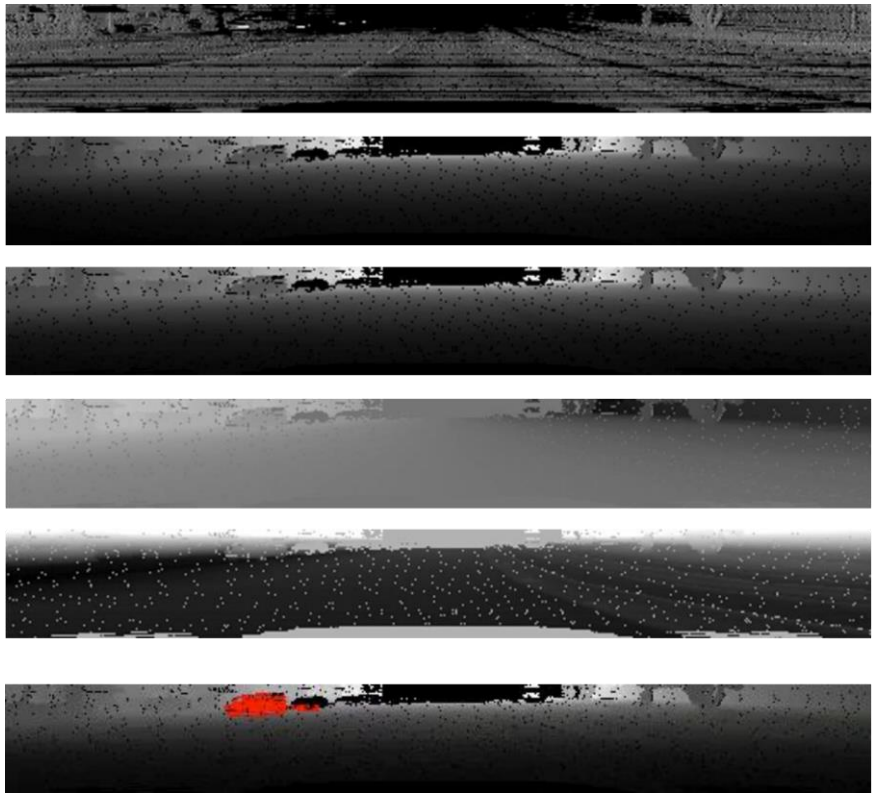
$\sqrt{x^2 + Y^2 + z^2}$

x

y

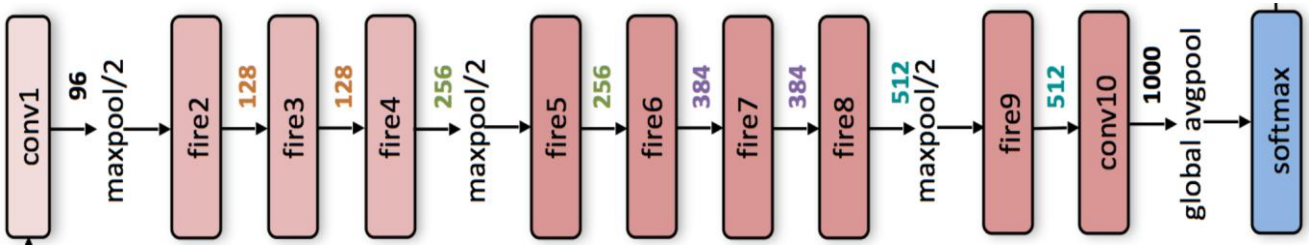
z

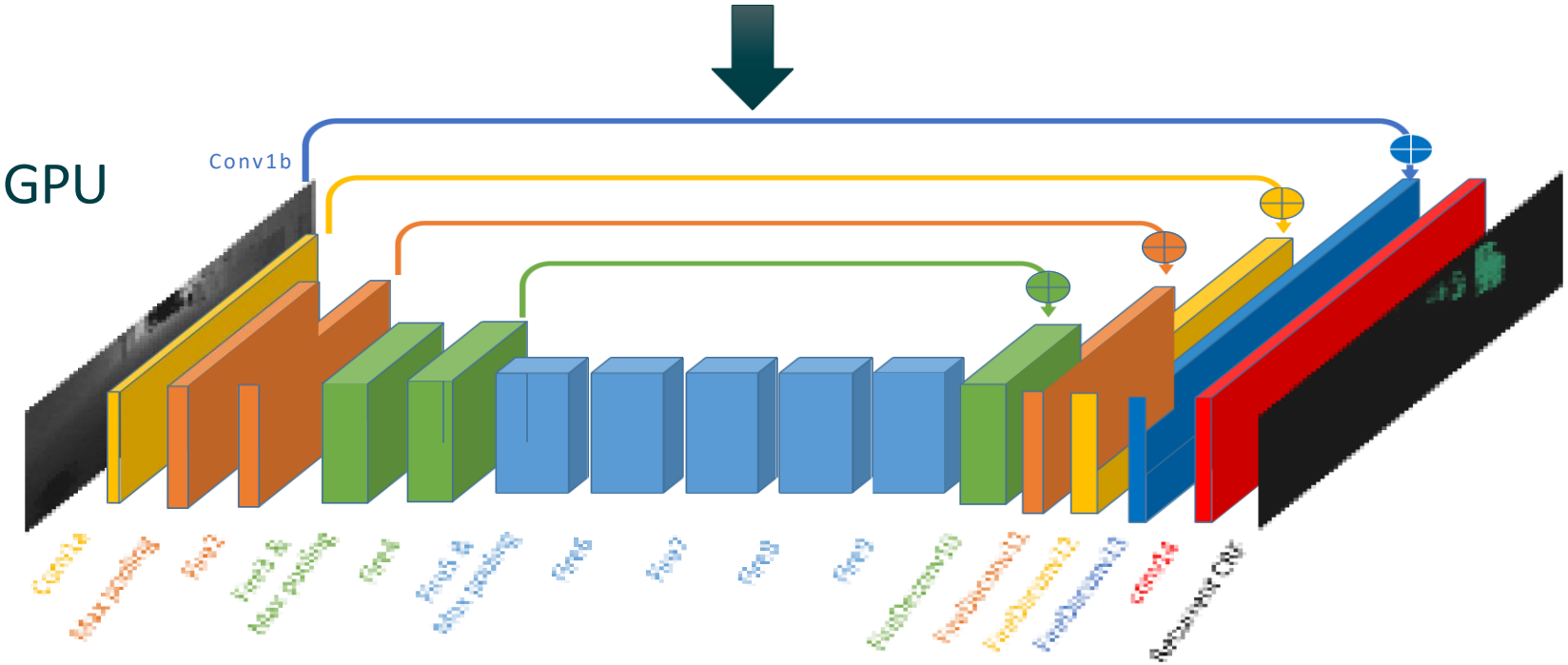
Label



CNN for Point Clouds

- Processing projected point cloud as 2D images
- Use an efficient 2D ConvNet (SqueezeNet) to predict point-wise labels
- Extremely fast:
  - > 25 FPS on an embedded GPU
  - > 100 FPS on desktop GPU





Understanding 3D Point Clouds

Video reference



Ground truth label map

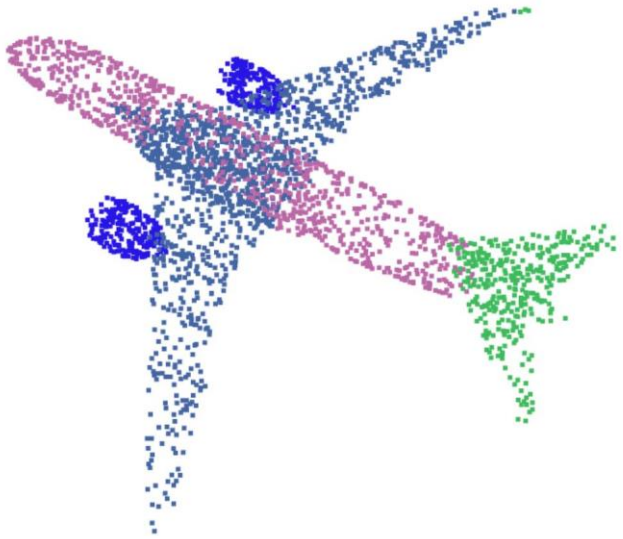


Predicted label map

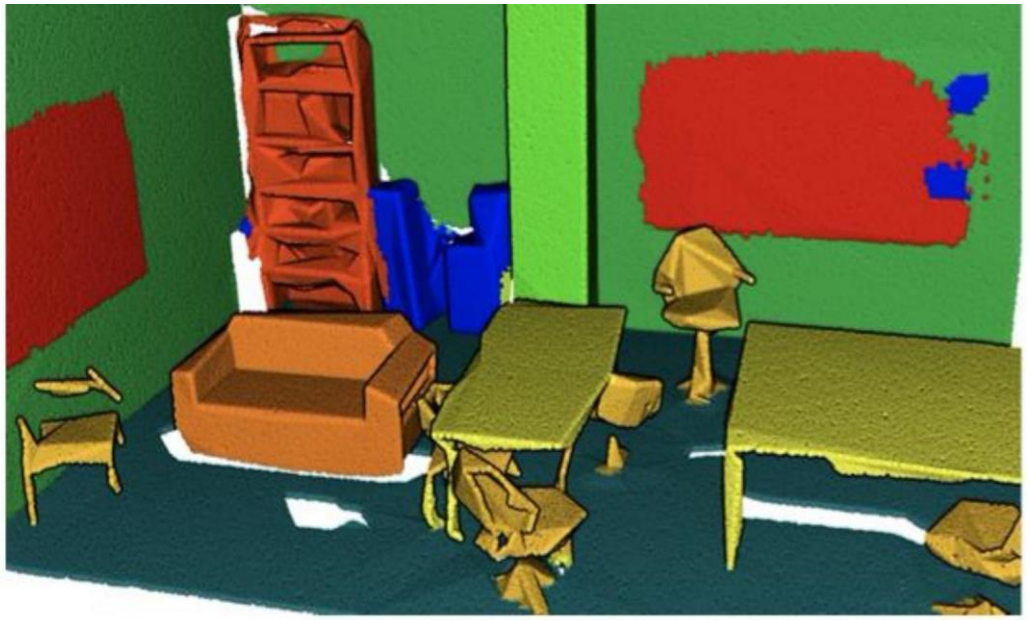


Point Clouds Unsuitable for 2D Projection

- Many point clouds cannot be conveniently projected to 2D
- Can we process point cloud directly as a set of points?



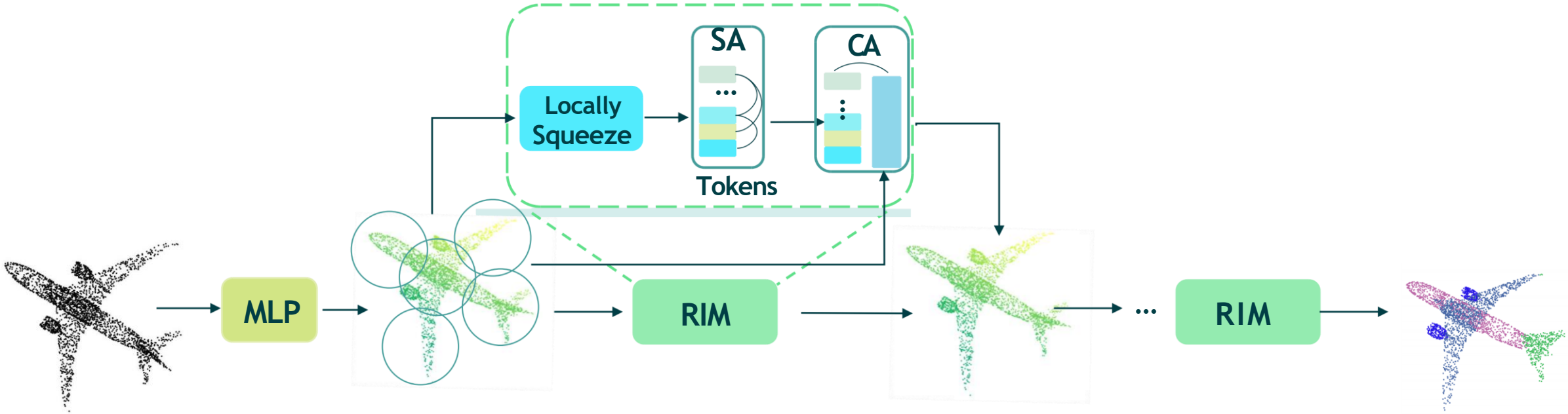
3D CAD models



Accumulated LiDAR scans

Transformer for Point Clouds: YOGO

- Divide a point cloud evenly into sub-regions using the farthest-point sampling
- Process each point using multi-layer perceptron (MLP), locally aggregate features
- Use self-attention to exchange information across local regions



SA: self-attention CA: cross attention RIM: Relation Inference Module

Understanding 3D Point Clouds

- Accuracy on-par with previous SOTA, but at least 3x faster

Method	Mean IoU	Latency	GPU Memory
PointNet [2]	83.7	21.4 ms	1.5 GB
RSNet [39]	84.9	73.8 ms	0.8 GB
SynSpecCNN [40]	84.7	-	-
PointNet++ [3]	85.1	77.7 ms	2.0 GB
PointNet++* [3]	85.4	236.7 ms	0.9 GB
DGCNN [41]	85.1	86.7 ms	2.4 GB
SpiderCNN [42]	85.3	170.1 ms	6.5 GB
SPLATNet [14]	85.4	-	-
SO-Net [33]	84.9	-	-
PointCNN [4]	86.1	134.2 ms	2.5 GB
YOGO (KNN)	85.2	25.6 ms	0.9 GB
YOGO (Ball query)	85.1	21.3 ms	1.0 GB

Method	Mean IoU	Latency	GPU Memory
PointNet [2]	42.97	24.8 ms	1.0 GB
DGCNN [41]	47.94	174.3 ms	2.4 GB
RSNet [39]	51.93	111.5 ms	1.1 GB
PointNet++* [3]	50.7	501.5 ms	1.6 GB
TangentConv [43]	52.6	-	-
PointCNN [4]	57.26	282.43 ms	4.6 GB
YOGO (KNN)	54.0	27.7 ms	2.0 GB
YOGO (Ball query)	53.8	24.0 ms	2.0 GB

Chenfeng Xu, et al. 2021

Lab6 Exercise (optional)

1. Read the provided papers of “detection and tracking”, “offline-tracking” as well as the master thesis
2. Try to run the open source code provided by two papers.
3. Submit a report with results and comments. The file extension is **pdf.**

To be continued